

Inheritance in Java

Inheritance is one of the core concepts of OOP (Object-Oriented Programming).

It allows one class to acquire properties and behaviors of another class.

Real-Life Example

Imagine:

- **Father** has:
 - house
 - land
 - bike

Son automatically gets access to them.

Same in Java:

- Parent class = existing properties/methods
- Child class = reuses them

Why Inheritance is Used

1. Code Reusability

Instead of writing same code again and again.

2. Maintainability

Update parent once → child also gets update.

3. Logical Relationship

IS-A relationship.

Example:

- Dog IS-A Animal
- Car IS-A Vehicle

Syntax of Inheritance

```
class Parent {
```

```
}
```

```
class Child extends Parent {
```

```
}
```

Keyword used:

extends

1. Single Level Inheritance

One child inherits one parent.

Structure

Parent

↓

Child

Example

```
class Animal {  
  
    void eat() {  
        System.out.println("Animal can eat");  
    }  
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog can bark");  
    }  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.eat(); // inherited  
        d.bark(); // own method  
    }  
}
```

Flow of Execution

Dog object created:

```
Dog d = new Dog();
```

Dog class checks:

- bark() → available
- eat() → not found

Then JVM goes to parent class:

- Animal class has eat()

So child can access it.

Memory Understanding

Dog object contains:

- Dog members
- inherited Animal members

Common Mistakes

Mistake 1

Trying to access child methods using parent reference.

```
Animal a = new Animal();  
a.bark(); // ERROR
```

Reason:

Parent doesn't know child-specific methods.

Mistake 2

Forgetting IS-A relationship.

Wrong:

```
class Engine extends Car
```

Engine is NOT a Car.

Use inheritance only for IS-A relationship.

2. Multi Level Inheritance

One class inherits another child class.

Structure

GrandParent

↓

Parent

↓

Child

Example

```
class Animal {  
  
    void eat() {  
        System.out.println("Animal can eat");  
    }  
}
```

```
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog can bark");  
    }  
}
```

```
class BabyDog extends Dog {
```

```
    void weep() {
```

```
        System.out.println("Baby dog can weep");
    }

    public static void main(String[] args) {

        BabyDog b = new BabyDog();

        b.eat();
        b.bark();
        b.weep();
    }
}
```

What Happens Internally

BabyDog gets:

From Dog:

- bark()

From Animal:

- eat()

Own:

- weep()

So inheritance chain is created.

Constructor Flow in Multi-Level Inheritance

When object created:

```
BabyDog b = new BabyDog();
```

Execution order:

Animal constructor

↓

Dog constructor

↓

BabyDog constructor

Parent constructor always executes first.

Common Mistakes

Deep Inheritance Chain

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Very difficult to maintain.

In frameworks this becomes dangerous.

Method Confusion

Too many inherited methods create confusion.

3. Hierarchical Inheritance

Multiple child classes inherit same parent.

Structure

```
Animal
 /  \
Dog   Cat
```

Example

```
class Animal {

    void eat() {
        System.out.println("Animal can eat");
    }
}

class Dog extends Animal {

    void bark() {
        System.out.println("Dog can bark");
    }
}

class Cat extends Animal {

    void meow() {
        System.out.println("Cat can meow");
    }
}

public class Main {

    public static void main(String[] args) {

        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

```
Cat c = new Cat();  
c.eat();  
c.meow();  
}  
}
```

Benefits

Common logic written once.

Both children reuse it.

Common Mistakes

Putting Unrelated Logic in Parent

Bad Example:

```
class Animal {  
    void fly() {}  
}
```

Now all animals get fly().

Wrong design.

4. Multiple Inheritance

VERY IMPORTANT TOPIC.

What is Multiple Inheritance?

One child inheriting from multiple parents.

Structure

```
Parent1
  \
   Child
  /
Parent2
```

Is Multiple Inheritance Possible with Classes in Java?

NO ❌

Java DOES NOT support multiple inheritance using classes.

Invalid Example

```
class A {}

class B {

}

class C extends A, B {

}
```

Compilation error.

Why Java Does NOT Support It?

Because of:

Diamond Problem

Example

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}
```

```
class B extends A {  
  
}
```

```
class C extends A {  
  
}
```

```
class D extends B, C {  
  
}
```

Now:

```
D d = new D();  
d.show();
```

Question:

Which show() should come?

From B side?

OR

From C side?

Ambiguity occurs.

So Java blocks it.

What is Possible in Inheritance?

Possible

1. Child inherits parent members

class B extends A

2. Method overriding

Child can redefine parent method.

3. Constructor chaining

Parent constructor automatically called.

4. Accessing protected members

Child can access protected members.

5. Using super keyword

Access parent constructor/method/variable.

What is NOT Possible ❌

1. Cyclic Inheritance

INVALID:

```
class A extends B {  
  
}
```

```
class B extends A {  
  
}
```

Why?

Infinite inheritance loop.

Java blocks it.

2. Multiple Inheritance using Classes

INVALID:

```
class C extends A, B
```

3. Private Members Inheritance

Private members are NOT directly accessible.

Example:

```
class Parent {  
  
    private int x = 10;  
}
```

```
class Child extends Parent {  
  
    void test() {  
        System.out.println(x); // ERROR  
    }  
}
```

Private belongs only to that class.

Parent Contains Custom Constructor

VERY IMPORTANT.

Example

```
class Parent {  
  
    Parent(int x) {  
        System.out.println("Parent Constructor");  
    }  
}
```

```
class Child extends Parent {  
  
    Child() {  
  
    }  
}
```

ERROR ❌

Why Error?

Compiler automatically adds:

```
super();
```

inside child constructor.

But parent has NO default constructor.

Only:

```
Parent(int x)
```

exists.

So compiler gets confused.

Correct Version

```
class Parent {
```

```
    Parent(int x) {  
        System.out.println("Parent Constructor");  
    }  
}
```

```
class Child extends Parent {
```

```
    Child() {  
        super(10);  
    }
```

```
    public static void main(String[] args) {
```

```
        Child c = new Child();  
    }  
}
```

Important Rules About Constructors in Inheritance

Rule 1

Parent constructor executes first.

Rule 2

Constructor is NOT inherited.

Methods are inherited.

Constructors are NOT.

Rule 3

Every constructor has:

`super();`

implicitly.

Rule 4

`super()` must be first statement.

INVALID:

```
Child() {  
    System.out.println("Hi");  
    super();  
}
```

Impact of Static Members on Inheritance

Static Variables

Inherited but belong to class.

Example

```
class Parent {  
  
    static int x = 10;  
}  
  
class Child extends Parent {  
  
    public static void main(String[] args) {  
  
        System.out.println(x);  
    }  
}
```

Works.

But memory belongs to Parent class.

Static Methods

Inherited.

But they are:

Hidden, not overridden.

Example

```
class Parent {  
  
    static void show() {  
        System.out.println("Parent");  
    }  
}
```

```
class Child extends Parent {  
  
    static void show() {  
        System.out.println("Child");  
    }  
}
```

This is:

Method Hiding

NOT overriding.

Static Blocks

NOT inherited directly.

But parent static block executes when class loads.

Example

```
class Parent {  
  
    static {
```

```
        System.out.println("Parent Static Block");
    }
}

class Child extends Parent {

    static {
        System.out.println("Child Static Block");
    }

    public static void main(String[] args) {

    }
}
```

Output:

Parent Static Block
Child Static Block

Most Common Inheritance Mistakes in Real Projects

1. Using Inheritance for Code Reuse Only

Wrong design.

Prefer composition sometimes.

2. Very Deep Inheritance

Hard to debug.

3. Forgetting Constructor Chaining

Causes compile errors.

5. Accessing Private Members Directly

Not possible.

Use getters/setters.

6. Using Static Methods Like Overridden Methods

Static methods are hidden, not overridden.

Final Important Summary

Feature	Supported?
Single Inheritance	✓
Multilevel Inheritance	✓
Hierarchical Inheritance	✓
Multiple Inheritance with Classes	✗
Multiple Inheritance with Interfaces	✓

Constructor Inherited	✗
Method Inherited	✓
Private Members Directly Accessible	✗
Static Method Overriding	✗
Method Overriding	✓
Cyclic Inheritance	✗